

01 — Run and debug Python scripts in VS Code / 在 VS Code 中运行与调试 Python 脚本

1. Run the example / 先运行一次

Open the repository folder in VS Code. Use **Ctrl+Shift+P** → **Python: Select Interpreter** to select an installed Python environment. The interpreter executes your code. You need the Microsoft Python and Python Debugger extensions.

在 VS Code 中打开仓库文件夹，按 **Ctrl+Shift+P** → **Python: Select Interpreter**，选择已有 Python 环境。解释器负责执行代码。需要 Microsoft 的 Python 和 Python Debugger 扩展。

Open the [chapter 10 script](#), save it, then choose **Run Python File in Terminal** from the upper-right triangle. This example needs Python 3.9+ and only the standard library. See [VS Code: running Python](#)^[1].

打开并保存第 10 章脚本，在右上角三角按钮中选择 **Run Python File in Terminal**。这个示例需要 Python 3.9+，只用标准库。参见 [VS Code: 运行 Python](#)^[1]。

Expected output:

预期输出：

```
p value: {'rho': 50.0}
p type: dict
annotation: dict[str, float] | None
annotation type: str
output path: None
```

Alternatively, run this in a PowerShell terminal at the repository root:

也可以在仓库根目录的 PowerShell 终端中运行：

```
python examples/10_python_annotations_and_parsing.py
```

If the prompt is `>>>`, enter `exit()` first to leave the Python interactive session.

如果提示符是 `>>>`，先输入 `exit()`，退出 Python 交互环境后再执行终端命令。

2. Pause and inspect a function / 暂停并观察函数

2.1 Do I need to enter arguments first? / 需要先输入实参吗？

No extra input is needed for the current example: the caller already supplies the argument. These excerpts are from the chapter 10 script:

当前案例不需要额外输入：调用处已经提供了实参。以下片段来自第 10 章脚本：

```

from __future__ import annotations

# Function definition
def identity(p: dict[str, float] | None = None):
    return p

# Inside main()
p = {"rho": 50.0}
value = identity(p)

```

A **parameter** is the receiving name in a function definition; an **argument** is what the caller supplies. Here, `p` receives the dictionary. Because its default is `None`, calling `identity()` returns `None`. Required parameters without defaults must receive arguments; the debugger does not prompt for them.

形参 (parameter) 是函数定义中接收值的名称，**实参 (argument)** 是调用时提供的对象。这里 `p` 接收字典；有默认值 `None`，所以调用 `identity()` 会返回 `None`。没有默认值的必需形参，调用时必须提供实参，调试器不会弹窗让你补填。

Here, `--output-dir` is an optional command-line argument, separate from the function argument. Omitting it leaves `args.output_dir` as `None`; `identity` still receives the dictionary. The script does not call `input()` or wait for keyboard input.

这里的 `--output-dir` 是可选的命令参数，与函数实参分开。不传该选项时，`args.output_dir` 为 `None`，而 `identity` 仍然接收字典。脚本没有调用 `input()`，也不会等待键盘输入。

2.2 Follow the argument into the function / 跟着实参进入函数

Keep the example `.py` file active. Click the gutter next to `value = identity(p)` to add a breakpoint. Start debugging with `F5`; if prompted, choose **Python Debugger** and the current Python file. If an existing launch configuration targets another program, choose the current-file configuration instead. See [Python debugging](#) ^[2].

保持示例 `.py` 文件为当前编辑文件。在 `value = identity(p)` 行号旁的空白处点击，添加断点。按 `F5` 开始调试；如出现选择提示，选择 **Python Debugger** 和当前 Python 文件。若已有启动配置指向其他程序，应改选当前文件配置。参见 [Python 调试](#) ^[2]。

At the breakpoint, the highlighted statement has not executed yet. On this fresh run, `p` is `{'rho': 50.0}`, while `value` has not been assigned. Press **Step Into** (`F11`) to enter `identity`. At `return p`, its local parameter `p` refers to that same dictionary.

停在断点时，高亮语句还未执行。在本次新启动的运行中，`p` 已经是 `{'rho': 50.0}`，但 `value` 尚未赋值。按 **Step Into / 单步进入** (`F11`)，进入 `identity`；停在 `return p` 时，函数的局部参数 `p` 指向同一个字典。

```

p          # {'rho': 50.0}
type(p)    # <class 'dict'>
p["rho"]   # 50.0

```

Enter these expressions separately in Debug Console while paused inside `identity`. Terminal is for shell commands or keyboard input requested by the program. They inspect the argument already received; they do not supply a missing argument. Use **Shift+F11** to return to the caller, then **F5** to continue after inspecting the completed assignment.

在 identity 内部暂停时，将这些表达式逐条输入 Debug Console。Terminal 则用于终端命令或程序要求的键盘输入。它们查看的是已经收到的实参，不是在补传参数。用 **Shift+F11** 返回调用处，核对赋值完成后的结果，再按 **F5** 继续运行。

Windows default action Windows 默认操作	What it does 作用
F10 — Step Over 单步跳过	Execute the current line without entering its function calls; other breakpoints can still interrupt. 执行当前行，不逐行进入其中的函数；其他断点仍可能使程序暂停。
F11 — Step Into 单步进入	Enter a called function when supported by the debugger. 在调试器支持时进入被调用函数。
Shift+F11 — Step Out 单步跳出	Finish the current function and return to its caller. 执行完当前函数，返回调用它的位置。
F5 — Continue 继续	Resume until another breakpoint, a configured exception stop, or completion. 继续到下一断点、已配置的异常暂停或程序结束。
Shift+F5 — Stop 停止	End the debugging session. 结束本次调试。

3. Prepare inputs for a debugging session / 调试时如何准备输入

First identify how the program receives its input. For this learning example, start with a fixed input in main() so that you can repeat the same run and compare values.

先确定程序通过什么方式接收输入。对于这个学习案例，建议先在 main() 中写好固定输入，方便重复运行并对照数值。

Input source 输入来源	How to supply it while debugging 调试时如何提供
A function call such as identity(test_p) 函数调用，例如 identity(test_p)	Create test_p before the call. 调用前准备好 test_p。
A command-line option such as --output-dir 命令行选项，例如 --output-dir	Set args in the selected launch.json configuration. 在所选 launch.json 配置的 args 中填写。
An interactive input() call input() 交互输入	Type in Terminal when the running program requests input. 程序提示输入时，在 Terminal 中键入。

3.1 Fixed arguments in a small caller / 在调用处准备固定实参

This is a standalone teaching variant, not a replacement for the chapter 10 script. It uses only the standard library. Save it as a separate practice .py file if you want to run it:

下面是只使用标准库的独立教学变体，供另存为练习 .py 文件后运行；第 10 章原脚本仍保留原样：

```
def identity(p=None):
    return p

def main():
    test_p = {"rho": 60.0}
    result = identity(test_p) # Set a breakpoint here
    print(result)

if __name__ == "__main__":
    main()
```

Start this practice file under the debugger, pause at the call, and press F11. In main, the argument is named `test_p`; inside `identity`, the receiving parameter is named `p`. The names can differ. Both refer to the same dictionary, and the final output is `{'rho': 60.0}`. Change `test_p` and restart to try a different input. Calling `identity()` instead returns `None` because this function has a default.

调试这个练习文件，停在函数调用处，再按 F11。在 `main` 中传入的变量名是 `test_p`，进入 `identity` 后接收它的形参名是 `p`，两者不必同名。它们指向同一个字典，最后输出 `{'rho': 60.0}`。修改 `test_p` 后重新启动调试，即可尝试其他输入。若改成调用 `identity()`，由于该函数有默认值，结果为 `None`。

3.2 Command-line inputs in launch.json / 在 launch.json 中设置命令行参数

For the existing chapter 10 script, the following configuration supplies the option `--output-dir results/demo`. This is a configuration example; documenting it does not change your VS Code settings. In `.vscode/launch.json`, add the configuration object to the existing configurations list, or use the whole file below if none exists:

对于已有的第 10 章脚本，下列配置传入 `--output-dir results/demo` 选项。这是配置示例，整理进教程不代表已修改你的 VS Code 设置。在 `.vscode/launch.json` 中，将配置对象加入已有的 `configurations` 列表；尚无该文件时，可使用下面的完整内容：

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Chapter 10: arguments",
      "type": "debugpy",
      "request": "launch",
      "program": "${workspaceFolder}/examples/10_python_annotations_and_parsing.py",
      "cwd": "${workspaceFolder}",
      "console": "integratedTerminal",
      "args": ["--output-dir", "results/demo"]
    }
  ]
}
```

Open the repository root as the VS Code workspace. Select **Chapter 10: arguments** in Run and Debug, then press F5. `program` fixes the script, `cwd` fixes the working directory, and `args` supplies script options. The final output is output path: `results/demo`; no directory is created. Each token is a separate `args` item. See [Python debugging](#) ^[2].

以仓库根目录作为 VS Code 工作区。在 Run and Debug 中选择 **Chapter 10: arguments**，再按 F5。`program` 指定脚本，`cwd` 指定工作目录，`args` 提供脚本选项。最后输出 output path: `results/demo`，不会创建目录。选项名和选项值分别占一个 `args` 元素。参见 [Python 调试](#) ^[2]。

This configuration always targets chapter 10, even when a different file is open. Switch back to a current-file configuration to debug the practice variant in section 3.1. `--output-dir` does not set `p` or `rho`: the script only connects this option to `args.output_dir`. Passing a value into `identity` still requires a Python function call.

即使当前打开其他文件，这份配置仍运行第 10 章脚本。调试第 3.1 节练习文件时，应切换回当前文件配置。`--output-dir` 不会设置 `p` 或 `rho`；脚本只把该选项解析到 `args.output_dir`。向 `identity` 传值仍需通过 Python 函数调用。

3.3 Interactive keyboard input / 键盘交互输入

For an interactive variant of section 3.1, keep its `identity` definition and entry-point guard, but replace the body of `main()` with:

若将第 3.1 节改成交互输入版本，保留 `identity` 定义和入口判断，将 `main()` 的函数体替换为：

```
rho = float(input("Enter rho: "))
p = {"rho": rho}
value = identity(p) # Set a breakpoint here
print(value)
```

Debug that practice file with the integrated terminal. When the prompt appears, select **Terminal**, type `60`, and press Enter. The program converts the text to `60.0`, builds the dictionary, and then reaches the breakpoint. Entering expressions in Debug Console does not answer this terminal prompt. Nonnumeric text fails at `float(...)` with `ValueError`, before the function call.

使用集成终端调试该练习文件。出现输入提示后，切换到 **Terminal**，输入 `60` 并回车。程序将文本转换为 `60.0`、创建字典，然后到达断点。在 Debug Console 输入表达式不会回答这个终端提示。若输入非数字文本，会在 `float(...)` 处触发 `ValueError`，尚未执行到函数调用。

Verification and next discussion / 核对与后续讨论

The original script and both practice variants were checked with Python 3.13.9. Fixed input and interactive input `60` both printed `{'rho': 60.0}`. The default call `identity()` returned `None`. The JSON example was parsed and its configured program and arguments were run from the repository root; the VS Code F5 launch itself was not performed.

原脚本和两个练习变体均已用 Python 3.13.9 核对。固定输入和交互输入 `60` 均输出 `{'rho': 60.0}`。默认调用 `identity()` 返回 `None`。JSON 示例已解析检查，并在仓库根目录执行了它指定的脚本与参数；本次未实际在 VS Code 中按 F5 启动该配置。

Next, we can add exception breakpoints and traceback reading based on your own debugging session. [01-SI](#) preserves the more detailed Spyder array-inspection example.

接下来可以结合你的实际调试会话，补充异常断点与错误回溯（traceback）的阅读方法。[01-SI](#)保留详细的 Spyder 数组检查案例。

参考资料 (References)

1. <https://code.visualstudio.com/docs/python/run>
2. <https://code.visualstudio.com/docs/python/debugging>